# REVIEW 1: PROJECT REPORT

### Agentic Shell — Smart Archive Agent

| | |
|---|---|
| **Course / Project:** Operating Systems Project — Agentic Shell | **Project Title:** Smart Archive Agent |
| **Team Details:** Team 12 | **Team Members:** Kush Aghera (24BCE0149), Sandeep More (24BCE2883) |

## Chapter 1: Introduction

### 1.1 Project Overview

#### The evolution of Command Line Interfaces (CLIs)

Command-line interfaces have been the main way to control Unix and Linux systems since the first UNIX shell, and shells such as Bash and Zsh are still what most administrators use every day. They are fast, they can be scripted, and they give direct control over files, processes and storage. The catch is that the user has to remember the exact command, its flags and the order of its arguments.

#### The shift toward Natural Language Processing (NLP) in system administration

Large Language Models have changed what a terminal can look like. Instead of remembering flags, a user can describe what they want, and an agent turns that into real system operations. Our class is building one such "Agentic Shell": a base shell sends each request to one of 35 specialised agents. Our part is Agent #12, the Smart Archive Agent, which handles archiving, compression, extraction, inspection and encryption on Ubuntu Linux.

### 1.2 Problem Statement

#### Syntactical barriers and steep learning curves of traditional shells (Bash/PowerShell)

Linux has several archive and compression tools (tar, gzip, bzip2, xz, zip and gpg), and each one has its own flags. An xz tarball needs tar -cJf, gzip needs -czf and bzip2 needs -cjf. xz and gzip delete the original file unless you remember -c, and encrypting with GPG adds --symmetric and --cipher-algo AES256. Unless you use these tools every day, you end up looking the flags up every time.

#### Risk of executing incorrect commands due to human error

Mistakes here can lose data. If the source and destination are swapped in a tar command, tar can overwrite the source file with an empty archive; we reproduced this ourselves (Appendix A, Figure A.2). Extracting an untrusted archive can also write files outside the target folder (the "Zip Slip" problem) or overwrite existing files. Our agent avoids this by only calling a fixed set of functions that check every path first and never overwrite anything.

## 1.3 Objectives

- **Lightweight NL-to-CLI translation:** turn plain-English requests into archive operations without heavy libraries or an internet connection.
- **Predefined operation boundaries:** map every request to one of eight fixed tools (create, extract, list, inspect, compress, decompress, encrypt, decrypt) instead of running shell commands typed by the user.
- **MCP-style tool design:** describe each tool with a name, a description and a JSON Schema, the way the Model Context Protocol describes tools, so the class shell's LLM can call our agent.
- **Security and guardrails:** check every path, block Zip Slip, never overwrite files, and provide AES-256 GPG encryption for sensitive files.
- **Structured feedback:** always return a clear result: what was created, what is inside an archive, or what went wrong.

## 1.4 Scope and Limitations

- **Supported operating systems:** Ubuntu Linux, including Ubuntu under WSL 2 on Windows, which is what we develop on. The agent uses the standard tools tar, gzip, bzip2, xz, file and gpg, and Python (written for 3.8 or newer, tested on 3.14); zip files are handled by Python's built-in zipfile module.
- **Terminal environments:** it runs in a normal terminal (Bash or Zsh), either as an interactive prompt or as a one-line command.
- **Administrative privileges:** it runs as a normal user and never asks for sudo, so it cannot damage system files.
- **Domain boundaries:** it only handles archiving, compression and encryption. Anything else, such as process management or network settings, is rejected so that the shell can send it to the right agent.

# Chapter 2: Literature Review & Theoretical Background

## 2.1 Large Language Models (LLMs) in Code Generation

### Capabilities of modern LLMs in structured text and code synthesis

Current LLM families such as OpenAI GPT, Anthropic Claude and Google Gemini are good at turning plain sentences into structured output. With function calling, a model fills in a predefined JSON schema instead of writing free text, which is exactly what an agent needs to pick a tool and its arguments.

### Challenges with LLM hallucination in system-level execution

The problem is that LLMs can hallucinate: they may invent file paths, make up flags or pick a destructive command. Running a model's output directly in bash is therefore unsafe. That is why we kept understanding the request separate from running it: whatever produces the tool call, every argument is checked by our own validation code before anything touches the system.

## 2.2 Introduction to Model Context Protocol (MCP)

### Architecture of MCP (Clients, Servers, and Tools)

The Model Context Protocol (MCP), published by Anthropic, is an open standard for connecting AI applications to tools and data. It has three parts: clients (the application that uses tools), servers (programs that offer tools) and tools (single functions, each described with a JSON Schema for its inputs).

### Why MCP is preferred over traditional rigid API tooling for local system access

Before MCP, connecting a model to local tools usually meant reading command output as plain text or writing a custom adapter for every tool. MCP gives one standard way to discover tools, check their inputs and report errors. Because we describe our tools in the same way, our agent works on its own today and can be plugged into the class-wide shell without changing its archive and security code.

## 2.3 Existing Solutions

We compared our approach with the terminal assistants that already exist:

| Tool | How it works | Safety | What it means for our project |
|---|---|---|---|
| Plain Bash | The user types every command by hand | Only normal user permissions; nothing checks the syntax | The tools themselves are reliable, but you must remember every flag |
| Warp AI | Terminal with AI command suggestions | Suggests shell commands; the user has to review them before running | Shows that plain-English terminals work, but it still runs whatever shell script it suggests |
| GitHub Copilot CLI | Turns a sentence into a shell command | Explains the command and asks the user to confirm | Shows LLM command generation, but the model can still make up flags |
| Smart Archive Agent (ours) | One agent focused only on archives | Fixed tools only; every path checked; Zip Slip blocked; never uses root | Plain-English requests, but every action goes through our own checks |

# References

[1] Anthropic, 'Model Context Protocol (MCP) Specification,' 2024. [Online]. Available: https://modelcontextprotocol.io/

[2] OpenAI, 'Function Calling and Structured Outputs Guide,' 2024. [Online]. Available: https://platform.openai.com/docs/guides/function-calling

[3] Free Software Foundation, 'GNU Tar Manual: Tape Archiving Utility,' GNU Project, 2023. [Online]. Available: https://www.gnu.org/software/tar/manual/

[4] Warp Technologies, 'Warp: The AI-Powered Terminal User Guide,' 2024. [Online]. Available: https://docs.warp.dev/

[5] GitHub Inc., 'GitHub Copilot in the CLI Documentation,' 2024. [Online]. Available: https://docs.github.com/en/copilot/using-github-copilot-in-the-cli

# Appendix A: Evidence of Work — Review 1 Stage

This appendix shows what we did in the first stage of the project: setting up our environment, checking the problem for ourselves, and creating the repository with the first version of the code. Everything below is real output from our machine, and our repository is public at https://github.com/redrighthand2007/OS-SMART-ARCHIVE-AGENT, so every commit can be checked.

## A.1 Our Development Environment

We develop on Ubuntu running inside WSL 2 on Windows, using the archive and encryption tools that come with Ubuntu. These are the versions we used:




**Figure A.1: Operating system, kernel, Python and tool versions on our development machine.**

## A.2 Checking the Problem with the Raw Linux Tools

Before writing any code, we wanted to see the problem from Section 1.2 for ourselves, so we tried the common mistakes in a test folder:

- **Swapping tar's arguments loses data:** report.txt held some text. We ran tar -cJf report.txt backup.tar.xz, with the source and destination the wrong way round. tar printed an error, but by then it had already replaced report.txt with an empty xz archive, so the text was gone.
- **xz removes the original:** after xz notes.txt, only notes.txt.xz is left.
- **The correct manual steps are long:** doing the brief's task by hand (archive heavily, encrypt, delete the plaintext) took us three commands and eight flags (XZ_OPT=-9e, -cJf, --batch, --pinentry-mode, --passphrase, --symmetric, --cipher-algo AES256, rm). With our agent it is one sentence.

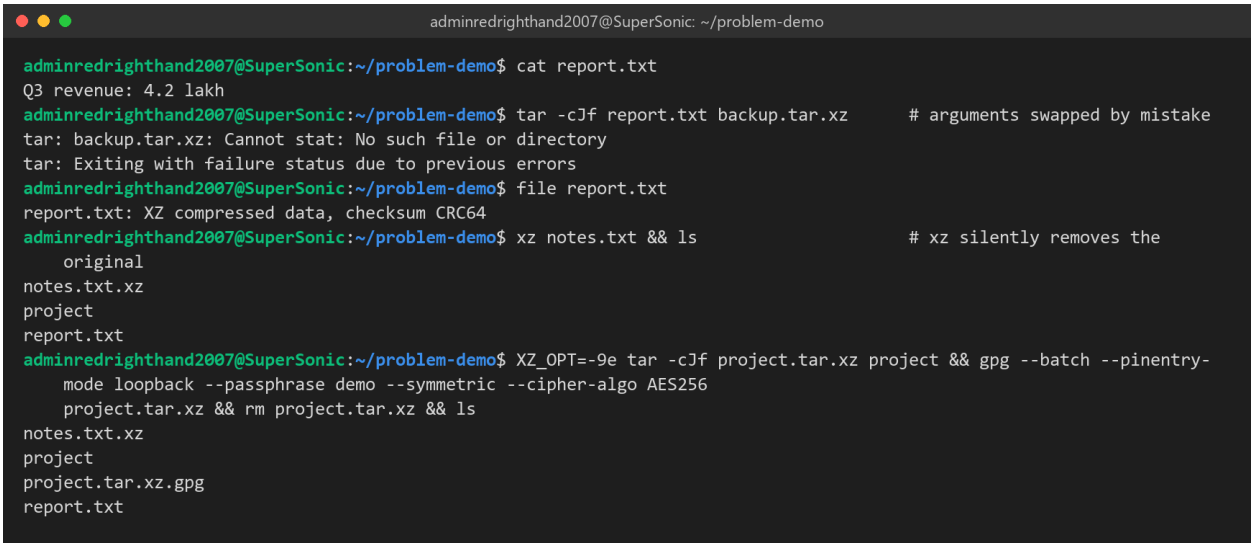



**Figure A.2: Data lost by swapped tar arguments, xz deleting its input, and the manual commands needed for the brief's task.**

## A.3 Repository and First Commits (Stage 1)

We created the GitHub repository and committed the project base, the first agent core, the first tests and the documentation. Both of us committed to the repository:

**Table A.1: Our Stage 1 commits (from git log).**

| Commit | Author | What we changed | Size |
|---|---|---|---|
| **e1f4d65** | KUSH AGHERA | Initial commit | 1 file, +1 / −0 lines |
| **49b7061** | KUSH AGHERA | INITIALIZED | 1 file, +6 / −1 lines |
| **eb8542b** | Kush Aghera | init project base | 3 files, +81 / −0 lines |
| **a2401f6** | Kush Aghera | add agent core | 9 files, +1773 / −0 lines |
| **d7cfe53** | Kush Aghera | add test suite | 6 files, +398 / −0 lines |
| **1912a9e** | Kush Aghera | add documentation files | 4 files, +325 / −4 lines |
| **759ba9c** | KUSH AGHERA | started | 1 file, +1 / −1 lines |
| **aca62bb** | Sandeep-More-1610 | Update member details for Sandeep More | 1 file, +1 / −1 lines |
| **d15e083** | Kush Aghera | add review docs | 2 files (binary: .docx / .pdf / .png) |
| **2d64067** | Kush Aghera | add review pdfs | 2 files (binary: .docx / .pdf / .png) |
| **ff40930** | Kush Aghera | add review three | 2 files (binary: .docx / .pdf / .png) |